

SAVEETHA SCHOOL OF ENGINEERING SIMATS UNIVERSITY

Name: Hitesh C

Register No: 192524005

Course Code: CSA1401

Course Name: COMPILER DESIGN

Faculty Name: Dr. V. AJITHA

ASSESSMENT TOOL 3

Technical Presentation

Q1. Phases of a Compiler and Their Role

Introduction

A compiler is system software that translates a high-level programming language into machine code. It performs this translation through several well-defined phases. Each phase performs a specific task and passes its output to the next phase, making compilation accurate, efficient, and modular.

Objectives

- Understand the phases of a compiler.
- Explain the role of each phase.
- Study error detection during compilation.
- Understand symbol table and error handler.
- Know the importance of modular compiler design.

Phases of a Compiler

Phase	Function	Output
Lexical Analysis	Converts characters into tokens.	Tokens
Syntax Analysis	Checks grammar using CFG.	Parse Tree
Semantic Analysis	Checks meaning and types.	Annotated Tree
Intermediate Code Generation	Creates intermediate representation.	IR
Code Optimization	Improves efficiency.	Optimized IR
Code Generation	Produces target machine code.	Executable Code

Explanation of Each Phase

1. Lexical Analysis

Reads source code character by character, removes spaces/comments and groups lexemes into tokens. It reports lexical errors such as invalid identifiers.

2. Syntax Analysis

Checks whether the sequence of tokens follows the grammar rules and constructs a parse tree.

3. Semantic Analysis

Checks variable declarations, data types, scope rules and type compatibility.

4. Intermediate Code Generation

Produces machine-independent intermediate code such as three-address code.

5. Code Optimization

Removes redundant operations, dead code and performs constant folding to improve execution.

6. Code Generation

Converts optimized intermediate code into assembly or machine code.

Compiler Flow

Source Program



Lexical Analysis



Syntax Analysis



Semantic Analysis



Intermediate Code Generation



Code Optimization



Code Generation



Target Program

Example

Source Code:

```
int a=5,b=10;
```

```
int c=a+b;
```

Lexical Analyzer identifies keywords, identifiers, operators and constants. Syntax Analyzer validates the statement. Semantic Analyzer checks declarations and types. Intermediate code is generated, optimized and finally translated to machine code.

Advantages

- Modular compilation process
- Easy error detection
- Improves execution speed
- Produces efficient machine code
- Supports multiple programming languages

Conclusion

The compiler consists of six major phases working together to convert source code into executable machine code. Each phase has a unique responsibility that improves correctness, efficiency and maintainability.

Q2. Lexical Analysis and Token Specification using LEX

Introduction

Lexical Analysis is the first phase of a compiler. It reads the source program character by character and converts it into meaningful tokens. LEX is a tool used to automatically generate lexical analyzers from token specifications written using regular expressions.

Objectives

- Understand lexical analysis.
- Learn tokens, lexemes and patterns.
- Study regular expressions.
- Understand LEX structure.
- Generate tokens using LEX.

Basic Terminology

Term	Meaning
Token	Smallest meaningful unit such as keyword or operator.
Lexeme	Actual character sequence matched.
Pattern	Rule that describes a set of lexemes.
Regular Expression	Expression used to define token patterns.

Role of Lexical Analyzer

- Reads source code.
- Removes spaces and comments.
- Identifies identifiers, keywords, literals and operators.
- Generates tokens for the parser.
- Reports lexical errors.

Token Specification

Identifier : [A-Za-z][A-Za-z0-9_]*

Number : [0-9]+

Keyword : if | else | while | int | float

Operator : + - * / == < >

Structure of a LEX Program

```
%{  
C declarations  
%}  
Definitions  
%%  
Rules and Actions  
%%  
User Functions
```

Sample LEX Program

```
%{  
#include<stdio.h>  
%}  
%%  
[0-9]+ printf("NUMBER\n");  
[a-zA-Z]+ printf("IDENTIFIER\n");  
"+" printf("PLUS\n");  
%%  
int yywrap(){ return 1; }
```

Working Flow

Source Program



Lexical Analyzer (LEX)



Token Generation



Syntax Analyzer

Advantages

- Fast token generation.
- Reduces manual coding.
- Easy maintenance.
- Supports regular expressions.
- Automatically detects lexical errors.

Applications

- Compiler construction
- Programming language development
- Source code analyzers
- Interpreters
- Text processing tools

Conclusion

LEX simplifies lexical analyzer development by automatically generating efficient scanners from regular-expression based token specifications. It improves compiler development speed and accuracy.

Q3. Language Processors: Compiler vs Interpreter

Introduction

Language processors convert source programs into executable form. The two most common language processors are the Compiler and the Interpreter. A compiler translates the entire source program before execution, whereas an interpreter translates and executes one statement at a time.

Objectives

- Understand language processors.
- Differentiate compiler and interpreter.
- Study advantages and disadvantages.
- Know real-world examples.
- Identify suitable applications.

Compiler

A compiler translates the complete source program into machine code before execution. If compilation is successful, an executable file is generated which can be run multiple times.

Interpreter

An interpreter translates and executes one statement at a time. No separate executable file is produced. Errors are reported immediately when encountered.

Compiler vs Interpreter Comparison

Feature	Compiler	Interpreter
Translation	Entire program	One statement at a time
Execution Speed	Fast	Slower
Error Reporting	After compilation	Line by line
Executable File	Generated	Not generated
Memory Usage	More	Less
Examples	C, C++, Go	Python, JavaScript, Ruby

Working Flow

Compiler:

Source Program



Compiler



Object Code



Executable Program

Interpreter:

Source Program



Interpreter



Immediate Execution

Example

C Program:

```
int main(){  
    printf("Hello");  
}
```

The C compiler first translates the entire program and then executes it.

Python Program:

```
print("Hello")
```

The Python interpreter translates and executes the statement immediately.

Advantages of Compiler

- Faster execution.
- Optimized machine code.
- Suitable for large applications.
- Executable can run without source code.
- Better performance.

Advantages of Interpreter

- Easy debugging.
- Immediate execution.
- No compilation step.
- Platform independent in many environments.
- Ideal for scripting.

Applications

Compilers are used for system software, operating systems, games, and embedded systems. Interpreters are commonly used in scripting, web development, automation, artificial intelligence, and data science.

Conclusion

Both compilers and interpreters are essential language processors. Compilers provide high performance and optimized execution, while interpreters offer flexibility, easy debugging, and rapid program development.

Q4. Grouping of Compiler Phases and Optimization

Introduction

A compiler is divided into different phases that work together to convert a source program into machine code. These phases are grouped into the Front End and Back End. Optimization is an important stage that improves program performance without changing its output.

Objectives

- Understand the grouping of compiler phases.

- Study front-end and back-end compiler architecture.
- Learn code optimization techniques.
- Know the advantages of optimization.
- Understand real-world applications.

Grouping of Compiler Phases

Group	Compiler Phases	Purpose
Front End	Lexical, Syntax, Semantic Analysis	Checks correctness of source code
Middle End	Intermediate Code Generation & Optimization	Improves efficiency
Back End	Code Generation	Produces machine code

Front-End Compiler

The front end analyzes the source program. It performs lexical analysis, syntax analysis, and semantic analysis. It detects errors and generates an intermediate representation.

Middle-End Compiler

The middle end generates intermediate code and performs optimization techniques such as constant folding, dead code elimination, common subexpression elimination, and loop optimization.

Back-End Compiler

The back end converts optimized intermediate code into assembly or machine code. It performs instruction selection, register allocation, and machine-specific optimizations.

Compiler Architecture Flow

Source Program



Front End

(Lexical + Syntax + Semantic)



Intermediate Code



Middle End (Optimization)



Back End (Code Generation)



Target Program

Code Optimization Techniques

Technique	Description
-----------	-------------

Constant Folding	Evaluates constant expressions during compilation.
Dead Code Elimination	Removes unused statements.
Loop Optimization	Improves loop execution speed.
Common Subexpression Elimination	Avoids repeated calculations.
Strength Reduction	Replaces expensive operations with cheaper ones.

Example

Before Optimization:

```
x = 10 * 2;
y = x + 0;
```

After Optimization:

```
x = 20;
y = x;
```

Advantages

- Faster program execution.
- Reduced memory usage.
- Efficient CPU utilization.
- Better overall performance.
- Improved executable quality.

Applications

Compiler optimization is widely used in operating systems, embedded systems, mobile applications, database systems, game engines, scientific computing, and artificial intelligence software.

Conclusion

Grouping compiler phases into front end, middle end, and back end makes compiler design modular and easier to maintain. Optimization techniques improve execution speed, reduce memory usage, and generate efficient machine code.

Q5. Compiler Construction Tools and LEX

Introduction

Compiler construction tools simplify the development of compilers by automating various phases such as lexical analysis, parsing, code generation, and optimization. LEX is one of the most popular tools used to generate lexical analyzers.

Objectives

- Understand compiler construction tools.
- Study the purpose of LEX.
- Learn the structure of a LEX program.
- Understand the working of LEX.
- Know the applications of compiler tools.

Compiler Construction Tools

Tool	Purpose	Example
Lexical Analyzer Generator	Creates lexical analyzers	LEX / Flex
Parser Generator	Generates syntax analyzers	YACC / Bison
Code Generator	Produces target code	LLVM
Optimization Tool	Improves generated code	LLVM Optimizer
Debugger	Finds program errors	GDB

What is LEX?

LEX is a lexical analyzer generator developed for UNIX systems. It reads token specifications written using regular expressions and automatically generates a C program that recognizes tokens.

Structure of a LEX Program

%{

C declarations

%}

Definitions

%%

Rules and Actions

%%

User Functions

Sample LEX Program

%{

#include<stdio.h>

%}

%%

[0-9]+ printf("NUMBER");

```
[a-zA-Z]+ printf("IDENTIFIER");  
"+"      printf("PLUS");  
%%  
int yywrap()  
{  
    return 1;  
}
```

Working of LEX

Input Source Program



LEX Specification



LEX Generator



Lexical Analyzer (C Program)



Token Generation

Advantages

- Automatic scanner generation.
- Supports regular expressions.
- Reduces manual coding effort.
- Easy to modify.
- Reliable and efficient.

Applications

- Compiler design
- Programming language development
- Source code analysis
- Data validation
- Pattern matching
- Text processing utilities

Conclusion

Compiler construction tools reduce development time and improve reliability. LEX is an important tool that automates lexical analysis and is widely used in compiler and interpreter development.